

मोबाइल एप विकास

BEGINNER TO ADVANCED PRACTICE

JavaScript · React Native · Expo

अध्ययन दिन	कुल अभ्यास	दैनिक समय
112 दिन	336 घण्टा	3 घण्टा

हरेक दिन के पढ्ने, कति अभ्यास गर्ने र काम पूरा भयो भनेर कसरी जाँच्ने - एउटै सन्दर्भ पुस्तिकामा।

तपाईंको लक्ष्य

Android र iOS का लागि एउटै React Native project बाट app बनाउने; APIs, offline data, device features, authentication, AI र release प्रक्रिया अभ्यास गर्ने।

नेपाली व्याख्या • JavaScript मुख्य भाषा • भिडियो अनिवार्य छैन

आधार: तपाईंले पठाउनुभएको १९-section course outline र तपाईंको React/JavaScript अनुभव। समय योजना अभ्यासको अनुमान हो; checkpoint अनुसार गति मिलाउनुहोस्।

तयार गरिएको: २६ सेप्टेम्बर २०२६ | Version 1.0

START HERE

यो योजना कसरी पढ्ने?

दिनको ३ घण्टा नियमित दिँदा ११२ अध्ययन दिनमा ३३६ घण्टा हुन्छ। सातै दिन पढे १६ हप्ता; हप्तामा ६ दिन पढे करिब १९ हप्ता लाग्छ। कठिन विषयमा १४-२८ थप दिन राख्न सकिन्छ।

दैनिक काम	मिनेट	के गर्ने?
समझना जाँच	15	हिजोको code नहेरी अवधारणा/flow लेख्ने।
पढाइ	35	आजको १-२ relevant doc sections पढ्ने; ५ मुख्य बुँदा टिप्ने।
Code अभ्यास	100	सानो example चलाउने, फेरबदल गर्ने र project मा जोड्ने।
Debug / test	20	कम्तीमा एउटा failure वा edge case जाँच्ने।
नोट / Git	10	के सिकियो, error र अर्को कदम लेख्ने; सानो commit।

साताको सातौँ दिन

३० मिनेट पुनरावृत्ति + ११० मिनेट project/catch-up + ३० मिनेट परीक्षण + १० मिनेट log। यो पनि ३ घण्टाकै अध्ययन दिन हो।

एक दिन सफल भयो भन्ने मापदण्ड

पढ्ने विषय बुझियो; आफ्नो हातले code लेखियो; आजको “जाँच” पूरा भयो; कम्तीमा ३ वाक्यमा flow बुझाउन सकियो। अझकिएको दिनलाई repeat गर्दा नम्बर अगाडि धकेल्नुहोस्।

भिडियो नहेरी सिक्ने क्रम

१. PDF बाट आजको Day र source code Rxx छान्नुहोस्।
२. Source मा नाम मिलेको heading पढेर सानो flow कोर्नुहोस्।
३. Example एकचोटि चलाएर बन्द गर्नुहोस्; फेरि आफैं बनाउनुहोस्।
४. Input/data/feature बदल्नुहोस् र error case जाँच्नुहोस्।
५. नबुझेको code वा error सहित प्रश्न सोध्नुहोस्।

ChatGPT सँग प्रयोग गर्ने वाक्य: “यो PDF को Day __ पढाऔँ। नेपालीमा अवधारणा, छोटो diagram, JavaScript example, अभ्यास र expected result दिनुहोस्।”

AT A GLANCE

१६ हप्ताको पूरा मार्ग

हप्ता	Day	मुख्य क्षेत्र	घण्टा
01	1-7	JavaScript, React र mobile को आधार	21
02	8-14	Core components र प्रयोग गर्न सजिलो UI	21
03	15-21	Navigation: React Navigation र Expo Router	21
04	22-28	API, pagination र data screens	21
05	29-35	Local storage र offline Notes Buddy	21
06	36-42	Camera, images, audio र device UX	21
07	43-49	Development build, sensors र maps	21
08	50-56	Connectivity, notifications र app lifecycle	21
09	57-63	Backend, database र वास्तविक authentication	21
10	64-70	OAuth, auth providers र app structure	21
11	71-77	Weather, wallpaper र interaction	21
12	78-84	AI Story app र सुरक्षित AI integration	21
13	85-91	Advanced engineering अभ्यास	21
14	92-98	Capstone: वास्तविक प्रयोगको app	21
15	99-105	Capstone hardening र user feedback	21
16	106-112	Build, updates, CI/CD र release	21

Foundation: Day 1-28 (84 घण्टा) | Intermediate: Day 29-70 (126 घण्टा)

Advanced अभ्यास: Day 71-91 (63 घण्टा) | Capstone + release: Day 92-112 (63 घण्टा)

PDF का weekly titles र reading links clickable छन्। अन्यमा project requirements, course coverage, sources र progress tracker छन्।

TOOLS & CURRENT GUIDANCE

पहिलो setup र version नियम

काम	प्रयोग गर्ने साधन
मुख्य भाषा	JavaScript / JSX; आवश्यक TypeScript example को logic JS मा बुझ्ने।
UI र navigation	React Native + StyleSheet; मुख्य app मा Expo Router।
Computer / phone	Windows + VS Code + Git + supported Node LTS; सुरुवात physical Android फोनमा।
Package manager	npm वा Bun मध्ये एक रोज्ने; एउटै project मा lockfile नमिसाउने।
Backend / data	परिचित Node/Express वा Next.js server; local SQLite; backend database अलग।
Testing / builds	Expo Go बाट सुरु; पछि dev build, Jest/component tests र EAS।

Day 6: न्यूनतम JavaScript app

```
node --version
npm --version
npx create-expo-app@latest mobile-lab --template blank
cd mobile-lab
npx expo start
```

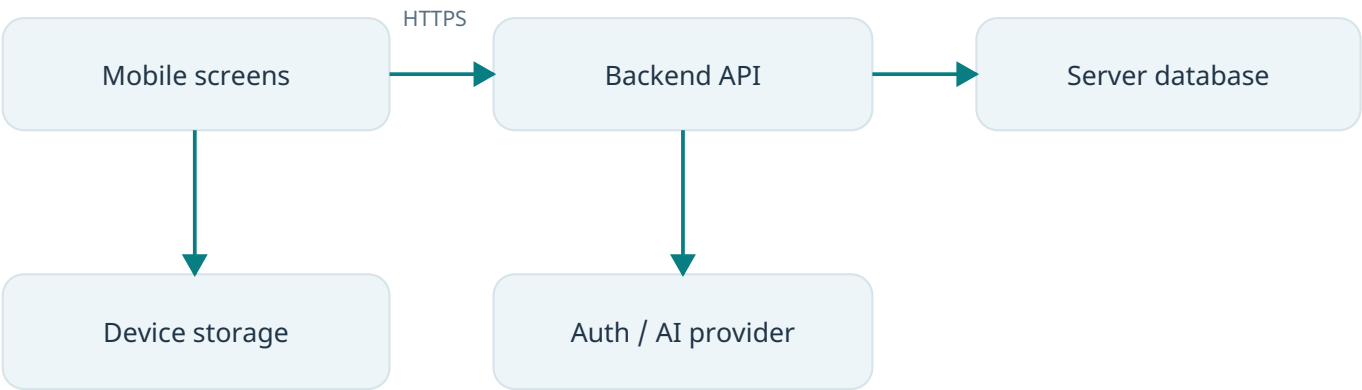
फोनमा compatible Expo Go खोल्नुहोस् र QR scan गर्नुहोस्। एउटै नेटवर्क प्रयोग गर्दा सजिलो हुन्छ। blank template मा सुरुको screen App.js हुन्छ; Router setup पछि entry संरचना बदलिन्छ। [R04, R06]

वर्तमान docs बाट ५ काम लाग्ने कुरा

- Expo Go आधार सिक्न उपयोगी छ; native integration का लागि dev build चाहिन्छ। [R13]
- OAuth र Android remote push को अभ्यास development build मा गर्नुहोस्। [R17, R18]
- Expo Maps अहिले alpha छ र Expo Go मा उपलब्ध छैन। [R14]
- SDK packages जोड्दा npx expo install प्रयोग गरी compatibility जाँच्नुहोस्।
- Lesson र SDK फरक भए आफ्नो installed SDK को docs छान्नुहोस्; audio/file APIs पुरानो code बाट जस्ताको तस्तै नलिनुहोस्। [R12, R29]

MENTAL MODEL

App भित्र data कहाँ जान्छ?



Local data supports offline use. Server validates access and secrets.

- १. Screen ले input लिन्छ र state अनुसार UI देखाउँछ।
- २. Local storage ले offline notes/preferences राख्छ।
- ३. Backend ले validation, authorization, sync र AI request चलाउँछ।
- ४. Server database ले account data राख्छ; API secret server मा रहन्छ।

Data	ठाउँ	उदाहरण
तुरुन्त बदलिने UI	useState / Context / Zustand	Input, modal open, selected filter
साधारण key-value	AsyncStorage	Theme, non-sensitive preference
सानो संवेदनशील value	SecureStore / auth SDK	आवश्यक session credential
Structured offline data	SQLite	Notes, reminders, sync queue
ठूला files	File System	Photo, audio, exported JSON
साझा account data	Backend database	Users, ownership, synced records

Protected screen ले UI नियन्त्रण गर्छ। Server ले हरेक request मा user र permission प्रमाणित गर्नुपर्छ। EXPO_PUBLIC variables app मा समावेश हुन्छन्; AI/provider secrets त्यहाँ नराख्नुहोस्। [R19]

READINESS CHECK

आधार जाँच र अगाडि बढ्ने नियम

तपाईं mobile development मा beginner हुनुहुन्छ; React/JavaScript को अनुभव भएको आधारमा पहिलो हप्ता revision राखिएको छ। यी काम सहज भए अतिरिक्त समय आफ्नै app मा लगाउनुहोस्।

Self-check	अपेक्षित काम
JavaScript	Array बाट नाम filter गर्ने; total reduce गर्ने; object immutable update गर्ने।
Async	API call मा loading, success र error छुट्याउने।
React	Props लिने component; state बदल्ने button; controlled input।
Effects	Timer/listener cleanup; dependency किन राखियो बताउने।
Git	Project commit; secret .env लाई repository बाट बाहिर राख्ने।

यी आधार कमजोर भए

पहिलो हप्ताको Day 2-5 मा थप ७-१४ दिन दिनुहोस्। JavaScript बिल्कुल नयाँ भए १४-२८ दिन prerequisite अभ्यास थप्न सकिन्छ। त्यही ३ घण्टाको routine कायम राख्नुहोस्।

हरेक topic कति गहिरो पढ्ने?

परिचय: के काम गर्छ र एउटा example। प्रयोग: आफूले feature बनाएर सामान्य तथा failure case जाँच्ने। गहिराइ: trade-off बुझाउने, test लेख्ने र performance/data behavior नाप्ने। Sensors र वैकल्पिक auth providers मा परिचय; navigation, API, storage, auth र capstone मा प्रयोग र गहिराइ।

अझैकिँदा अपनाउने क्रम

Error को पहिलो उपयोगी line पढ्ने □ expected/actual behavior लेख्ने □ सानो reproduction बनाउने □ package/SDK docs मिलाउने □ एकपटकमा एउटा परिवर्तन गर्ने □ फेरि जाँच्ने। ३० मिनेटपछि error, file र प्रयाससहित सहयोग माग्नुहोस्।

अध्ययनको अन्तिम क्षमता

चल्ने mobile app, आफ्नै backend integration, offline data, test evidence, release build र निर्णयहरूको व्याख्या। Advanced विषयमा नियमित वास्तविक project र feedback ले दक्षता थपिन्छ।

WEEK 01 / DAYS 1-7 / 21 HOURS

JavaScript, React र mobile को आधार

आफ्नो React ज्ञानलाई मोबाइलमा जोड्ने र फोनमा पहिलो app चलाउने।

पढाइका स्रोत: [R01 JavaScript Guide - MDN](#) | [R02 React Learn](#) | [R03 React Native documentation](#) | [R04 Expo project templates](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 01 सुरुवात र आफ्नो स्तर जाँच

पढ्ने: React Native, Expo, Android/iOS र native UI को सम्बन्ध। JS self-check का ५ काम।

बनाउने: आफ्नो लक्ष्य लेख्नुहोस्। array filter, total निकाल्ने function र immutable object update बनाउनुहोस्।

जाँच: ३ JS काम सही छन्; आफ्नो कमजोर २ विषय नोट छन्।

DAY 02 आवश्यक JavaScript

पढ्ने: Destructuring, spread, map/filter/reduce, modules; हरेकको एउटा उदाहरण।

बनाउने: १० वस्तुको shopping list बाट search, filter र total निकाल्ने ३ functions लेख्नुहोस्।

जाँच: खाली array र सामान्य array दुवैमा सही result आउँछ।

DAY 03 Async JavaScript

पढ्ने: Promise, async/await, try/catch, JSON र HTTP status का आधार।

बनाउने: एउटा public API call गर्ने function; success र failure अलग handle गर्नुहोस्।

जाँच: Data र error किन छुट्याउनुपर्छ आफ्नै शब्दमा लेख्न सक्नुहुन्छ।

DAY 04 React फेरि प्रयोग

पढ्ने: JSX, component, props, useState र event handler।

बनाउने: Counter र reusable ProfileCard बनाउनुहोस्; React web वा सानो अभ्यास project प्रयोग गर्न सक्नुहुन्छ।

जाँच: Parent बाट props जान्छ; button ले state बदल्छ।

WEEK 01 / Mobile Lab: counter, profile र form

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 05 Effects र cleanup

पढ्ने: useEffect dependencies, cleanup र custom hook को भूमिका।

बनाउने: Timer बनाउनुहोस्। screen/component हट्दा timer रोक्नुहोस्; useCounter hook निकाल्नुहोस्।

जाँच: अनावश्यक repeated effect हुँदैन; state update बुझाउन सक्नुहुन्छ।

DAY 06 Expo setup

पढ्ने: Node LTS, VS Code, Git, npm/Bun; blank JavaScript template र Expo Go।

बनाउने: Setup पृष्ठका commands चलाउनुहोस्; Android फोनमा app खोल्नुहोस् र greeting बदल्नुहोस्।

जाँच: फोनमा edit देखिन्छ; package manager र lockfile एउटा छ।

DAY 07 साताको अभ्यास र architecture निर्णय

पढ्ने: State कहाँ बस्छ र app, server, database को फरक दोहोर्याउनुहोस्।

बनाउने: Namaste counter app code नहेरी फेरि बनाउनुहोस्। ५ मिनेटमा React Native र Expo सम्बन्ध बुझाउनुहोस्।

जाँच: चल्ने app, screenshot, छोटो README र पहिलो Git commit तयार छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Code नहेरी counter बनाउनुहोस्; props र state को फरक भन्नुहोस्; app फोनमा चलोस्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 02 / DAYS 8-14 / 21 HOURS

Core components र प्रयोग गर्न सजिलो UI

Core components प्रयोग गरेर form, list र reusable screen बनाउन सक्ने।

पढाइका स्रोत: [R03 React Native documentation](#) | [R12 Expo SDK reference](#) | [R24 React Native accessibility](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 08 View, Text, Image, Pressable

पढ्ने: प्रत्येक component को काम, props र onPress; local र remote image।

बनाउने: Avatar, नाम, bio र follow button भएको profile बनाउनुहोस्; image नभए fallback राख्नुहोस्।

जाँच: Button प्रतिक्रिया दिन्छ; image error हुँदा screen बिग्रिँदैन।

DAY 09 StyleSheet र Flexbox

पढ्ने: StyleSheet, flexDirection, justifyContent, alignItems, padding र margin।

बनाउने: एक profile card का vertical र horizontal दुई layouts बनाउनुहोस्।

जाँच: हरेक प्रयोग गरेको style ले के बदल्छ भन्न सक्नुहुन्छ।

DAY 10 Responsive र platform UI

पढ्ने: useWindowDimensions, safe area, keyboard handling र Platform को परिचय।

बनाउने: सानो/ठूलो screen मा चल्ने login form बनाउनुहोस्; label र keyboardType थप्नुहोस्।

जाँच: Keyboard खुल्दा input र submit चल्छ; text screen बाहिर जाँदैन।

DAY 11 Controlled form र validation

पढ्ने: TextInput value/onChangeText; trim, required र error state।

बनाउने: नाम, email र note भएको ३-field form; submit गर्दा sanitized object देखाउनुहोस्।

जाँच: खाली नाम रोक्छ; गलत input सच्याउँदा error हट्छ।

WEEK 02 / Profile Gallery + Contacts

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 12 FlatList

पढ्ने: data, renderItem, keyExtractor, empty state र ScrollView सँग तुलना।

बनाउने: २० contacts देखाउनुहोस्; search र empty message थप्नुहोस्।

जाँच: Stable id प्रयोग भएको छ; नतिजा नभए स्पष्ट message आउँछ।

DAY 13 SectionList र theme

पढ्ने: Grouped lists, reusable components, light/dark theme र accessibility labels।

बनाउने: Contacts लाई A/B/C समूहमा राख्नुहोस्; दुई color themes बनाउनुहोस्।

जाँच: दुवै theme मा पढ्न मिल्छ; icon button को label छ।

DAY 14 Profile Gallery पूरा

पढ्ने: Components, forms र lists को पुनरावृत्ति।

बनाउने: Profile र Contacts लाई एउटै mini app मा जोड्नुहोस्; ३ UI bugs खोजेर सुधारनुहोस्।

जाँच: README, ३ screenshots र form/list को working demo तयार छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

२० contacts देखिन्छन्; search चल्छ; form error स्पष्ट छ; keyboard ले button छोप्दैन।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 03 / DAYS 15-21 / 21 HOURS

Navigation: React Navigation र Expo Router

Screens, params, nested layouts र mobile back behavior बुझ्ने।

पढाइका स्रोत: [R05 React Navigation](#) | [R06 Expo Router setup](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 15 Navigation को mental model

पढ्ने: Stack, tabs, drawer र modal को प्रयोग; React Navigation को आधार।

बनाउने: सानो React Navigation sandbox मा Home र Details राख्नुहोस्।

जाँच: Home बाट Details र back जान्छ; एउटै app मा duplicate root छैन।

DAY 16 Params र screen lifecycle

पढ्ने: Route params, focus र serializable data; id पठाउनुको अर्थ।

बनाउने: १० items मध्ये एउटा थिच्दा id अनुसार detail देखाउनुहोस्।

जाँच: पूरै object को सट्टा id पठाउने निर्णय बुझाउन सक्नुहुन्छ।

DAY 17 Expo Router setup

पढ्ने: File routes, entry point र _layout; JavaScript मा manual installation।

बनाउने: अलग router sandbox मा index.jsx र about.jsx बनाउनुहोस्; Link प्रयोग गर्नुहोस्।

जाँच: दुई routes खुल्छन्; मुख्य अध्ययन project को navigation Router मा छ।

DAY 18 Dynamic, catch-all र unmatched routes

पढ्ने: [id], [...slug] र +not-found को भूमिका।

बनाउने: Item detail, nested help path र unknown URL screen बनाउनुहोस्।

जाँच: Invalid id र unknown path दुवैका स्पष्ट fallback छन्।

WEEK 03 / Explore App: list, detail, favorites, settings

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 19 Layouts, Slot र route groups

पढ्ने: _layout, Slot, Stack र (group) ले structure कसरी मिलाउँछन्।

बनाउने: (tabs) group, detail Stack र settings screen मिलाउनुहोस्।

जाँच: Layout nesting र URL बीचको फरक भन्न सक्नुहुन्छ।

DAY 20 Tabs, drawer, modal, platform

पढ्ने: Tab layout, modal presentation, drawer basics र platform-specific file।

बनाउने: Favorites tab र filter modal थप्नुहोस्; अलग sandbox मा drawer चलाउनुहोस्।

जाँच: Android back ले modal बन्द गर्छ; navigation loop हुँदैन।

DAY 21 Navigation review

पढ्ने: Screen diagram र params को छोटो पुनरावृत्ति।

बनाउने: Explore App को flow code नहेरी चित्रमा बनाउनुहोस्; ५ navigation cases जाँच्नुहोस्।

जाँच: Home, detail, tab, modal र unmatched route pass छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Direct detail URL, back button, tab state र unknown route सबै चल्छन्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 04 / DAYS 22-28 / 21 HOURS

API, pagination र data screens

Network data लाई loading, error, empty र success states सहित देखाउने।

पढाइका स्रोत: [R03 React Native documentation](#) | [R07 Expo Router API routes](#) | [R19 Expo environment variables](#) | [R31 PokéAPI reference](#) | [R32 NewsAPI documentation](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 22 Fetch र API client

पढ्ने: HTTP, response.ok, JSON र base URL; फोनको localhost फरक हुन्छ।

बनाउने: Pokémon सूची ल्याउनुहोस्; API functions लाई services folder मा राख्नुहोस्।

जाँच: Loading र error छुट्टै देखिन्छ; failed request मा retry छ।

DAY 23 List र details

पढ्ने: List response र detail response; id र URL parameters।

बनाउने: List item खोल्दा detail API call गरेर नाम, image र ३ attributes देखाउनुहोस्।

जाँच: Unknown item वा missing image ले crash गर्दैन।

DAY 24 Pagination

पढ्ने: Page/offset/cursor को अर्थ; hasMore र loading guard।

बनाउने: Load More बाट अर्को page ल्याउनुहोस्; एउटै item फेरि नथपिने बनाउनुहोस्।

जाँच: छिटो दुईचोटि थिच्दा duplicate request/data आउँदैन।

DAY 25 Infinite scroll र refresh

पढ्ने: onEndReached, pull-to-refresh, debounce र cleanup।

बनाउने: Infinite scroll थप्नुहोस्; refresh गर्दा पहिलो page र नयाँ data देखाउनुहोस्।

जाँच: Refresh पछि पुराना page data दोहोरिँदैनन्।

WEEK 04 / Pokémon Explorer; News Reader सानो अभ्यास

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 26 News Reader अभ्यास

पढ्ने: Nested JSON, missing fields र API response inspect गर्ने तरिका।

बनाउने: NewsAPI वा उपलब्ध news feed बाट headline list बनाउनुहोस्; credential नभए fixture JSON प्रयोग गर्नुहोस्।

जाँच: Fixture वा live mode README मा स्पष्ट छ; loading/error/empty states छन्।

DAY 27 Expo Router API routes

पढ्ने: +api route, client/server code र deployed server को आवश्यकता।

बनाउने: hello endpoint र एउटा validated POST endpoint बनाउनुहोस्; mobile बाट call flow लेख्नुहोस्।

जाँच: Server कहाँ चल्छ भन्न सक्नुहुन्छ; production मा server deploy गर्नुपर्ने बुझिएको छ।

DAY 28 API project review

पढ्ने: Loading/error/empty/success state matrix बनाउनुहोस्।

बनाउने: Pokémon Explorer पूरा गर्नुहोस्; airplane mode, slow response र retry जाँच्नुहोस्।

जाँच: ५ API scenarios र pagination चलेको evidence README मा छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

Slow network, failure, duplicate page र unknown item हुँदा app स्थिर रहन्छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 05 / DAYS 29-35 / 21 HOURS

Local storage र offline Notes Buddy

Data को प्रकार हेरेर AsyncStorage, SecureStore, SQLite वा file रोज्ने।

पढाइका स्रोत: [R09 AsyncStorage](#) | [R10 SecureStore](#) | [R11 SQLite](#) | [R12 Expo SDK reference](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 29 Storage छनोट र AsyncStorage

पढ्ने: Key-value, JSON stringify/parse; theme र favorites जस्ता साधारण data।

बनाउने: Theme र favorites save/load गर्नुहोस्; malformed JSON को fallback राख्नुहोस्।

जाँच: Restart पछि theme फर्किन्छ; secrets यसमा राखिएको छैन।

DAY 30 SecureStore

पढ्ने: सानो संवेदनशील value को storage; read/write/delete र error handling।

बनाउने: Demo token राख्ने, पढ्ने र मेटाउने sandbox बनाउनुहोस्।

जाँच: Logout simulation मा token हट्छ; वास्तविक secret log हुँदैन।

DAY 31 SQLite आधार

पढ्ने: Table, row, primary key, SELECT/INSERT र parameterized queries।

बनाउने: notes table बनाउनुहोस्; title/body/time सहित ३ notes save गर्नुहोस्।

जाँच: App restart पछि पनि ३ notes पढ्न सकिन्छ।

DAY 32 SQLite CRUD

पढ्ने: UPDATE, DELETE, WHERE र validation।

बनाउने: Add, edit, delete र list screens जोड्नुहोस्; delete confirmation राख्नुहोस्।

जाँच: चारै CRUD काम र खाली title रोक्ने logic चल्छ।

WEEK 05 / Notes Buddy v1

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 33 Search, migration र transaction

पढ्ने: Schema version, safe migration, transaction र index को आधार।

बनाउने: पछिल्लो notes नमेटि pinned field थप्नुहोस्; search र pin filter बनाउनुहोस्।

जाँच: Migration अघि/पछि पुराना notes बाँकी छन्।

DAY 34 File System

पढ्ने: File/Directory, document/cache र file URI; JSON export/import।

बनाउने: Notes JSON मा export गर्नुहोस्; import मा shape validate गर्नुहोस्।

जाँच: गलत file ले database बिगाड्ने; current SDK API प्रयोग गरिएको छ।

DAY 35 Offline review र storage निर्णय

पढ्ने: चार storage विकल्पको तुलना आफ्नै उदाहरणमा।

बनाउने: Notes Buddy restart, airplane mode, invalid import र delete cases जाँच्नुहोस्।

जाँच: Offline CRUD, export/import र storage निर्णयको README तयार छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

App बन्द गरेर खोल्दा notes बाँकी छन्; airplane mode मा CRUD चल्ल।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्‍याउनुपर्ने Day: _____

WEEK 06 / DAYS 36-42 / 21 HOURS

Camera, images, audio र device UX

Permissions सहित device features जोड़ने र resources सफा गर्ने।

पढाइका स्रोत: [R12 Expo SDK reference](#) | [R24 React Native accessibility](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 36 Permission flow र camera

पढ्ने: Camera permission, denied state, retry/settings र camera lifecycle।

बनाउने: Photo capture screen; allow/deny दुवै UI बनाउनुहोस्।

जाँच: Permission deny गर्दा कारण र अगाडिको बाटो देखिन्छ।

DAY 37 Image picker र image display

पढ्ने: Library picker, cancelled result, expo-image र image dimensions।

बनाउने: Gallery बाट note मा photo जोड्नुहोस्; cancel गरे पनि पुरानो note सुरक्षित रहोस्।

जाँच: Photo देखिन्छ; missing/cancelled result handle भएको छ।

DAY 38 Image files

पढ्ने: Resize/compress, permanent location र temporary file को फरक।

बनाउने: ठूलो photo घटाउनुहोस्; note सँग local file URI save गर्नुहोस्।

जाँच: Restart पछि attachment देखिन्छ; file नभए fallback छ।

DAY 39 Audio playback

पढ्ने: expo-audio, player state, pause र resource cleanup।

बनाउने: एउटा local audio play/pause गर्ने छोटो screen बनाउनुहोस्।

जाँच: अर्को screen गएपछि चाहिएको behavior हुन्छ; दोहोरो playback छैन।

WEEK 06 / Notes Buddy: photo र voice attachments

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 40 Voice recording

पढ्ने: Microphone permission, recording start/stop र file save।

बनाउने: Voice note record गरेर note मा attach गर्नुहोस्; permission denied flow राख्नुहोस्।

जाँच: Record, stop र playback हुन्छ; note delete गर्दा file handling सोचिएको छ।

DAY 41 Haptics, battery र linking

पढ्ने: Haptic feedback, battery status, tel/mail/http links र failure handling।

बनाउने: Save feedback, battery label र call/open-link का साना प्रयोग बनाउनुहोस्।

जाँच: Unsupported feature को fallback छ; link खोल्न नसक्दा message आउँछ।

DAY 42 Media review

पढ्ने: Foreground/background र permission matrix दोहोर्याउनुहोस्।

बनाउने: Photo र voice सहित ३ notes बनाउनुहोस्; deny, cancel, restart र delete जाँच्नुहोस्।

जाँच: Media demo र ४ edge cases documented छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Permission नदिए पनि app usable छ; screen छोड्दा camera/audio बन्द हुन्छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 07 / DAYS 43-49 / 21 HOURS

Development build, sensors र maps

Expo Go बाट development build मा जाने र hardware availability जाँच्ने।

पढाइका स्रोत: [R13 Development builds](#) | [R14 Expo Maps](#) | [R15 Expo Sensors](#) | [R25 EAS Build setup](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 43 Development build र EAS

पढ्ने: Expo Go, dev client, preview र production build; native change हुँदा rebuild।

बनाउने: EAS project configure गरी Android development build बनाउनुहोस् र फोनमा install गर्नुहोस्।

जाँच: आफ्नै app icon बाट dev client खुल्छ; build log पढ्न सक्नुहुन्छ।

DAY 44 Accelerometer र gyroscope

पढ्ने: Sensor values, update interval, availability र listener cleanup।

बनाउने: X/Y/Z देखाउने screen र tilt हुँदा box सर्ने सानो खेल बनाउनुहोस्।

जाँच: Screen छोड्दा subscription हट्छ; unavailable sensor को message छ।

DAY 45 Magnetometer, light र pedometer

पढ्ने: Compass, light reading र step count; OS/device support फरक हुन सक्छ।

बनाउने: तीन sensor cards; supported sensor मात्र enable गर्नुहोस्।

जाँच: Data वा स्पष्ट unsupported state देखिन्छ; unavailable device मा fixture label छ।

DAY 46 Location

पढ्ने: Foreground permission, coordinates, accuracy र request timeout।

बनाउने: Current location लिएर latitude/longitude देखाउनुहोस्; manual location fallback राख्नुहोस्।

जाँच: Permission deny वा timeout हुँदा app चलिरहन्छ।

WEEK 07 / Device Lab + Places Map

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 47 Expo Maps परिचय

पढ्ने: Google Maps/Apple Maps, provider setup, development build र alpha status।

बनाउने: Current SDK docs हेरेर एउटा map र दुई markers बनाउनुहोस्; SDK नमिले react-native-maps विकल्प जाँच्नुहोस्।

जाँच: रोजेको library र platform support README मा छ; map देखिन्छ।

DAY 48 Map interaction

पढ्ने: Camera controls, zoom, compass/gestures, marker press र polyline support।

बनाउने: Marker थिच्दा detail card; supported API बाट दुई points को polyline बनाउनुहोस्।

जाँच: Marker र camera चल्छन्; polyline लाई वास्तविक route नमानिएको छ।

DAY 49 Device review

पढ्ने: Sensor/location/map permissions र lifecycle दोहोर्याउनुहोस्।

बनाउने: Places Map मा ३ saved places राख्नुहोस्; tilt game र sensor lab demo तयार गर्नुहोस्।

जाँच: Supported features tested छन्; बाँकी platform limitation स्पष्ट लेखिएको छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

Android development build चल्छ; unsupported sensor मा app crash गर्दैन; map marker खुल्छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 08 / DAYS 50-56 / 21 HOURS

Connectivity, notifications र app lifecycle

Internet र app state बदलिँदा सही प्रतिक्रिया दिने, reminder र push flow बनाउने।

पढाइका स्रोत: [R12 Expo SDK reference](#) | [R16 Network and Linking - SDK index](#) | [R17 Notifications](#) | [R13 Development builds](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 50 Network र connectivity

पढ्ने: Connection status, reachable server र actual request success अलग कुरा हुन्।

बनाउने: Offline banner र retry button; reconnect हुँदा data refresh गर्ने logic बनाउनुहोस्।

जाँच: Wi-Fi भए पनि API fail हुँदा error handle हुन्छ।

DAY 51 AppState र deep links

पढ्ने: Foreground/background, custom scheme र external URL validation।

बनाउने: Detail screen deep link बाट खोल्नुहोस्; resume हुँदा stale data refresh गर्नुहोस्।

जाँच: Invalid link सुरक्षित fallback मा जान्छ; event listeners cleanup छन्।

DAY 52 Local notifications

पढ्ने: Permission, notification channel, schedule र cancel।

बनाउने: केही मिनेटपछिको reminder schedule गर्नुहोस्; cancel button थप्नुहोस्।

जाँच: फोनमा notification आउँछ; cancel गरेको reminder आउँदैन।

DAY 53 Reminder app

पढ्ने: Date/time input, emoji/category, storage र timezone सोच।

बनाउने: Add/edit/delete reminder; note र notification id को सम्बन्ध राख्नुहोस्।

जाँच: Edit गर्दा पुरानो schedule हट्छ; delete गर्दा notification cancel हुन्छ।

WEEK 08 / Reminder Buddy

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 54 Push notifications setup

पढ्ने: Development build, Expo push token र platform credentials।

बनाउने: आफ्नो physical device token सुरक्षित backend record मा राख्नुहोस्; आफ्नै device मा test push पठाउनुहोस्।

जाँच: Token log/public repo मा छैन; permission denied case छ।

DAY 55 Push delivery र navigation

पढ्ने: Send, ticket/receipt, token invalidation र notification response।

बनाउने: Notification tap बाट सम्बन्धित note खोल्नुहोस्; invalid token handling को सानो backend flow बनाउनुहोस्।

जाँच: Foreground र background case छुट्टै जाँचिएका छन्; duplicate action हुँदैन।

DAY 56 Reminder review

पढ्ने: Local/push, device/server responsibilities दोहोर्याउनुहोस्।

बनाउने: ५ reminders र एउटा push flow जाँच्नुहोस्; अनुमति नदिएको case सहित README लेख्नुहोस्।

जाँच: Demo, persisted reminders र cancel/edit tests तयार छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Reminder schedule/cancel चल्छ; notification tap ले सही screen खोल्छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 09 / DAYS 57-63 / 21 HOURS

Backend, database र वास्तविक authentication

Mobile UI बाट protected backend data सम्मको flow आफैं बनाउन सक्ने।

पढाइका स्रोत: [R18 OAuth / OpenID authentication](#) | [R19 Expo environment variables](#) | [R10 SecureStore](#) | [R22 Expo unit testing + RN testing](#) | [R30 Express backend](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 57 Backend architecture

पढ्ने: Mobile, HTTPS API, server database, environment variables र API contract।

बनाउने: Express/Node वा परिचित Next.js backend रोज्नुहोस्; health र notes endpoints को contract लेख्नुहोस्।

जाँच: Mobile मा server secret छैन; status/response shapes स्पष्ट छन्।

DAY 58 Server data र validation

पढ्ने: User/note relationship, schema, parameterized query/ORM र server validation।

बनाउने: Database मा notes CRUD endpoints बनाउनुहोस्; गलत body लाई validation error दिनुहोस्।

जाँच: Postman मा create/read/update/delete चल्छ; invalid input reject हुन्छ।

DAY 59 Authentication आधार

पढ्ने: Identity, authorization, password hashing र session; SQLite demo को सीमा।

बनाउने: स्थापित auth library/service रोज्नुहोस्। local SQLite login simulation को जोखिम लेखेर server signup/login जोड्नुहोस्।

जाँच: Raw password database/phone/log मा छैन; local demo लाई real auth भनिएको छैन।

DAY 60 Mobile login र protected screens

पढ्ने: Login form, auth state, pending session र protected route।

बनाउने: App मा login/logout र protected notes screen जोड्नुहोस्।

जाँच: App launch मा session जाँचिँदै गर्दा गलत screen flash हुँदैन।

WEEK 09 / Account + Private Notes API

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 61 Session restore र expiry

पढ्ने: Secure session storage, expiry, refresh strategy र provider SDK behavior।

बनाउने: Restart पछि session restore; expired session मा re-login वा supported refresh राख्नुहोस्।

जाँच: Invalid token ले infinite retry गर्दैन; logout मा local credential हट्छ।

DAY 62 Authorization र account flows

पढ्ने: Owner check, rate limit, password reset/account deletion को design।

बनाउने: हरेक note operation मा owner check; दुई test users बाट access cases जाँचनुहोस्।

जाँच: User B को note id हाल्दा पनि access denied हुन्छ।

DAY 63 Authentication review

पढ्ने: Login/logout, restart, offline र expiry को test matrix।

बनाउने: ६ auth scenarios जाँचनुहोस्; phone/server/database flow आफैं diagram बनाउनुहोस्।

जाँच: Private notes API र mobile client चल्छन्; boundary स्पष्ट छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

User A ले User B को note पढ्न/बदल्न सक्दैन; logout र expired session ठीक छन्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 10 / DAYS 64-70 / 21 HOURS

OAuth, auth providers र app structure

OAuth flow बुझ्ने, provider तुलना गर्ने र auth logic सफा राख्ने।

पढाइका स्रोत: [R18 OAuth / OpenID authentication](#) | [R20 Better Auth Expo integration](#) | [R21 Clerk Expo quickstart](#) | [R22 Expo unit testing + RN testing](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 64 OAuth को flow

पढ्ने: Authorization code, PKCE, redirect URI, state र app scheme।

बनाउने: Browser/provider/app/backend को OAuth diagram बनाउनुहोस्; dev build redirect configure गर्नुहोस्।

जाँच: Client secret कहाँ रहन्छ र callback कसले जाँच्छ भन्न सक्नुहुन्छ।

DAY 65 Google sign-in

पढ्ने: Provider app setup, client type, allowed redirects र cancel/failure cases।

बनाउने: रोजेको auth solution को official guide अनुसार Google sign-in सानो sandbox मा जोड्नुहोस्।

जाँच: Success/cancel/error UI छन्; config नपुगे कुन भाग बाँकी हो स्पष्ट छ।

DAY 66 GitHub sign-in

पढ्ने: GitHub OAuth app, callback र server token exchange।

बनाउने: अलग sandbox मा GitHub login flow बनाउनुहोस्; mobile bundle मा secret नराख्नुहोस्।

जाँच: Backend/provider SDK ले session प्रमाणित गर्छ; profile मात्र trust गरिएको छैन।

DAY 67 Better Auth परिचय

पढ्ने: Expo integration, server/client plugin र secure storage adapter।

बनाउने: अलग branch मा minimal Better Auth email/session flow अध्ययन र sample चलाउनुहोस्; TS annotation को JS अर्थ छुट्याउनुहोस्।

जाँच: कुन code server र कुन client हो छुट्याउन सक्नुहुन्छ।

WEEK 10 / Auth Lab + मुख्य app को session integration

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 68 Clerk परिचय

पढ्ने: Provider, session hook, token cache र protected UI।

बनाउने: अलग sandbox मा Clerk quickstart को न्यूनतम flow बनाउनुहोस्; मुख्य app का लागि एउटा auth stack रोज्नुहोस्।

जाँच: Better Auth/Clerk/custom library तीनै stack एउटै app मा मिसाइएको छैन।

DAY 69 App structure र shared state

पढ्ने: features, components, services, hooks; Context/Zustand र server data cache।

बनाउने: Auth र notes logic screen बाट अलग गर्नुहोस्; global state आवश्यक ठाउँमा मात्र राख्नुहोस्।

जाँच: API call, UI र local DB logic छुट्टै जिम्मेवारीमा छन्।

DAY 70 Auth architecture review

पढ्ने: ३ user stories: signup, logout, expired session; trade-offs।

बनाउने: रोजेको provider का ३ integration tests र छोटो decision note लेख्नुहोस्।

जाँच: Auth working demo र छनोटको कारण तयार छ; बाँकी providers परिचयका रूपमा चिनिएका छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

एक auth provider राम्रोसँग integrate छ; Google/GitHub flow बुझिन्छ; credentials अलग छन्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 11 / DAYS 71-77 / 21 HOURS

Weather, wallpaper र interaction

Custom hooks, image-heavy lists र सहज interactions अभ्यास गर्ने।

पढाइका स्रोत: [R03 React Native documentation](#) | [R12 Expo SDK reference](#) | [R23 React Native performance](#) | [R24 React Native accessibility](#) | [R33 Open-Meteo weather API](#) | [R34 Pixabay API](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 71 Weather API

पढ्ने: Forecast/current data, units, coordinates र API response।

बनाउने: City वा coordinates बाट weather ल्याउने service function बनाउनुहोस्।

जाँच: Loading, unknown city र API failure अलग handle छन्।

DAY 72 Custom weather hook

पढ्ने: useWeather को input/output, effect cleanup र cache freshness।

बनाउने: Location र manual city input जोड्नुहोस्; hook बाट data/error/loading फर्काउनुहोस्।

जाँच: Permission deny हुँदा city input बाट app प्रयोग गर्न सकिन्छ।

DAY 73 Weather UI

पढ्ने: Condition-to-background mapping, readable text र reduced motion।

बनाउने: Sunny/rain/cloud का background र forecast cards; service response अनुसार UI बनाउनुहोस्।

जाँच: सजावटले text नछोपोस्; missing data मा fallback छ।

DAY 74 Pixabay wallpaper list

पढ्ने: Search/image API, paging, attribution र usage terms पढ्ने बानी।

बनाउने: २-column image list; search र loading placeholder बनाउनुहोस्।

जाँच: Missing image र no results ठीक छन्; स्रोत/terms note छ।

WEEK 11 / Weather Now + Wallpaper Explorer

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 75 Filter र image detail modal

पढ्ने: Controlled filters, debounce, modal dismissal र image sizes।

बनाउने: Category/filter modal र detail modal; full image खोल्ने flow बनाउनुहोस्।

जाँच: Filters reset चल्छ; Android back ले modal बन्द गर्छ।

DAY 76 Animation र gestures

पढ्ने: Animated/Reanimated को भूमिका, gesture feedback र unnecessary animation।

बनाउने: एउटा sheet वा card interaction animate गर्नुहोस्; reduce-motion fallback राख्नुहोस्।

जाँच: JS कामले interaction धेरै रोक्दैन; animation हटाउँदा पनि app usable छ।

DAY 77 Project review

पढ्ने: List performance र accessibility जाँचको आधार।

बनाउने: Weather र Wallpaper project का मुख्य flows test गर्नुहोस्; shared components निकाल्नुहोस्।

जाँच: दुवै mini apps का screenshots, README र error cases तयार छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Location deny हुँदा manual city चल्छ; wallpaper list र details सुरक्षित रूपमा खुल्छन्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 12 / DAYS 78-84 / 21 HOURS

AI Story app र सुरक्षित AI integration

AI features लाई backend, validation, storage र usage limits सहित जोड्ने।

पढाइका स्रोत: [R07 Expo Router API routes](#) | [R19 Expo environment variables](#) | [R22 Expo unit testing + RN testing](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 78 AI app design

पढ्ने: User input, server prompt, provider response, UI र storage को flow।

बनाउने: ३ screens को Story Pocket plan; age/topic/length input र expected response schema लेख्नुहोस्।

जाँच: MVP मा generate, read र save मात्र छन्; secret server मा बस्छ।

DAY 79 Story generation endpoint

पढ्ने: Backend validation, timeout, output shape र provider error।

बनाउने: एउटा text generation endpoint; सुरुमा fixture response, त्यसपछि उपलब्ध provider जोड्नुहोस्।

जाँच: Malformed input र provider failure मा controlled error आउँछ।

DAY 80 Mobile generation experience

पढ्ने: Submitting/loading, disable duplicate action, retry र cancellation।

बनाउने: Generate form र result screen; slow request मा progress message राख्नुहोस्।

जाँच: दुईचोटि थिच्दा अनावश्यक duplicate charge/request नहोस्।

DAY 81 Story storage

पढ्ने: Local SQLite/JSON file, ids, metadata र source of truth।

बनाउने: Generated stories device मा save/list/delete; server DB बिना local persistence देखाउनुहोस्।

जाँच: App restart पछि stories छन्; local data backup होइन भन्ने स्पष्ट छ।

WEEK 12 / Story Pocket

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 82 Image generation

पढ्ने: Server-side image job, result URL/file, delay र storage expiry।

बनाउने: एउटा cover-image flow; provider नभए labeled fixture प्रयोग गर्नुहोस्।

जाँच: Image fail भए पनि text story पढ्न मिल्छ; temporary URL permanence मानिएको छैन।

DAY 83 AI quality र limits

पढ्ने: Prompt boundaries, structured output check, rate limit र per-user usage cap।

बनाउने: १० inputs ले परीक्षण; server मा length/usage cap र sensitive logs घटाउनुहोस्।

जाँच: गलत output UI मा blindly render हुँदैन; key client bundle मा छैन।

DAY 84 AI project review

पढ्ने: Usefulness, latency, failure recovery र user feedback।

बनाउने: Story Pocket पूरा गर्नुहोस्; सफल, असफल र offline पढाइका demos राख्नुहोस्।

जाँच: AI अनुभवको निर्णय note, १०-case results र README तयार छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

AI request server बाट जान्छ; कथा बचत हुन्छ; failure/cancel मा प्रयोगकर्ता अड्किँदैन।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 13 / DAYS 85-91 / 21 HOURS

Advanced engineering अभ्यास

Sync, testing, performance र native boundary का आधार project मा प्रयोग गर्ने।

पढाइका स्रोत: [R22 Expo unit testing + RN testing](#) | [R23 React Native performance](#) | [R24 React Native accessibility](#) | [R13 Development builds](#) | [R35 React Native Testing Library](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 85 Offline sync design

पढ्ने: Local source of truth, pending queue, server id र conflict policy।

बनाउने: notes मा syncStatus/update time राख्नुहोस्; एउटा create operation queue गर्नुहोस्।

जाँच: Offline note बनाउँदा data हराउँदैन; pending state देखिन्छ।

DAY 86 Retry र conflicts

पढ्ने: Idempotency key, exponential backoff, duplicate prevention र conflict resolution।

बनाउने: Repeated request मा एउटै server record; दुई edits को सरल conflict rule बनाउनुहोस्।

जाँच: Reconnect गर्दा duplicate छैन; conflicting version silent overwrite हुँदैन।

DAY 87 Automated testing

पढ्ने: Jest, component behavior tests र API mocking; snapshot मात्र पर्याप्त हुँदैन।

बनाउने: Validation, note save र login failure का कम्तीमा ५ अर्थपूर्ण tests लेख्नुहोस्।

जाँच: उद्देश्यपूर्वक bug राख्दा सम्बन्धित test fail हुन्छ।

DAY 88 Performance

पढ्ने: Render profiling, FlatList virtualization, image size, memoization र release build।

बनाउने: ठूलो fixture list चलाउनुहोस्; एउटा bottleneck नापेर सुधार गर्नुहोस्।

जाँच: अघि/पछि result लेखिएको छ; अनुमानका आधारमा सबैमा memo छैन।

WEEK 13 / Notes Buddy v2: sync प्रयोग

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 89 Accessibility र localization

पढ्ने: Screen reader label, focus, font scaling, contrast र translated strings।

बनाउने: मुख्य flow screen reader बाट जाँचनुहोस्; Nepali/English text toggle राख्नुहोस्।

जाँच: ठूलो font मा text काटिँदैन; icon-only button को अर्थ सुनिन्छ।

DAY 90 Native boundary र architecture

पढ्ने: JS/native boundary, Hermes/Fabric/TurboModules को high-level अर्थ, config plugins र prebuild।

बनाउने: एउटा library को native setup पढ्नुहोस्; configuration change ले rebuild किन चाहिन्छ diagram बनाउनुहोस्।

जाँच: कहिले JS edit पुग्छ र कहिले native build चाहिन्छ छुट्याउन सक्नुहुन्छ।

DAY 91 Engineering review

पढ्ने: Sync failure, lifecycle, quality र deployment boundary दोहोर्याउनुहोस्।

बनाउने: १०-case regression run; unit/component/API tests र native boundary note तयार गर्नुहोस्।

जाँच: Critical flows pass छन्; बाँकी bugs priority अनुसार सूचीबद्ध छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

Offline edit reconnect पछि duplicate बिना sync हुन्छ; core tests र performance evidence छन्।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 14 / DAYS 92-98 / 21 HOURS

Capstone: वास्तविक प्रयोगको app

आफैं scope तय गरेर सिकेका features एक उत्पादनमा जोड्ने।

पढाइका स्रोत: R06 Expo Router setup | R10 SecureStore | R11 SQLite | R22 Expo unit testing + RN testing

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 92 Problem, scope र acceptance

पढ्ने: Target user, primary job, constraints र measurable done criteria।

बनाउने: VisitBook का ५ core stories लेख्नुहोस्; ६-screen wireframe र MVP feature list तय गर्नुहोस्।

जाँच: Core र पछि थप्ने features स्पष्ट छन्; acceptance checks लिखित छन्।

DAY 93 App foundation

पढ्ने: Feature folders, navigation, theme, services र environment config।

बनाउने: Home/list, add/edit, detail, reminders, login/settings screens को skeleton बनाउनुहोस्।

जाँच: सबै routes खुल्छन्; empty/loading/error components साझा छन्।

DAY 94 Offline core

पढ्ने: Data model, local SQLite repository र validation।

बनाउने: Visit/note create, edit, delete, search र filter जोड्नुहोस्।

जाँच: Airplane mode मा पूर्ण CRUD र restart persistence चल्छ।

DAY 95 Photo र location

पढ्ने: Attachment lifecycle, permission fallback र manually entered address।

बनाउने: Photo तथा optional location जोड्नुहोस्; permission बिना पनि visit save होस्।

जाँच: Deny/cancel/missing-file cases मा record सुरक्षित छ।

WEEK 14 / VisitBook: client visit/field notes; चाहिँदा अर्को समान scope

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 96 Follow-up reminders

पढ्ने: Reminder schedule/edit/cancel र record relation।

बनाउने: प्रत्येक visit मा next follow-up राख्नुहोस्; upcoming list बनाउनुहोस्।

जाँच: Visit edit/delete गर्दा reminder schedule पनि मिल्छ।

DAY 97 Auth र sync जोड्ने

पढ्ने: चयन गरिएको auth provider, ownership र साधारण sync queue।

बनाउने: Personal account सहित backup/sync को एउटा working path जोड्नुहोस्; pending/error UI राख्नुहोस्।

जाँच: User data अलग छ; failed sync ले local note हराउँदैन।

DAY 98 Capstone core review

पढ्ने: Core acceptance list र user journey।

बनाउने: Sample client का ५ visits बनाएर पूरा flow चलाउनुहोस्; blockers सुधारनुहोस्।

जाँच: Core demo तयार छ; build अर्को फोनमा परीक्षणका लागि तयार छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

एक वास्तविक काम end to end हुन्छ: visit record, photo, location, reminder र search।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 15 / DAYS 99-105 / 21 HOURS

Capstone hardening र user feedback

Main app लाई वास्तविक प्रयोगमा भरपर्दो बनाउने।

पढाइका स्रोत: R22 Expo unit testing + RN testing | R23 React Native performance | R24 React Native accessibility | R19 Expo environment variables

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 99 Test plan

पढ्ने: Unit, component, API र device end-to-end test को फरक।

बनाउने: १० critical cases र कम्तीमा ५ automated tests; अर्को install मा smoke test गर्नुहोस्।

जाँच: Known failing critical test छैन; tested/untested platform स्पष्ट छन्।

DAY 100 Data र session robustness

पढ्ने: App kill, request timeout, retry, token expiry र deleted records।

बनाउने: Save समयमा app बन्द, offline reconnect र expired session simulate गर्नुहोस्।

जाँच: Data loss र duplicate जस्ता major bugs सुधारिएका छन्।

DAY 101 Useful optional feature

पढ्ने: AI summary वा CSV/JSON export मध्ये एकको value र complexity तुलना।

बनाउने: समय बाँकी भए visit summary वा data export थप्नुहोस्; core bugs भए यही समय सुधारमा लगाउनुहोस्।

जाँच: Feature optional भनेर scope मा छ; core reliability घटेको छैन।

DAY 102 Device UX र accessibility

पढ्ने: Keyboard, safe area, large text, contrast र Android back।

बनाउने: दुई screen sizes र सबै forms जाँचनुहोस्; label/focus सुधारनुहोस्।

जाँच: मुख्य काम ठूलो text र keyboard सहित पूरा हुन्छ।

WEEK 15 / VisitBook release candidate

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 103 Performance र error reporting

पढ्ने: Release profiling, slow screen, large image र sanitized error logs।

बनाउने: Load/save समय नाप्नुहोस्; एउटा slow path सुधारनुहोस्; error report मा secret हटाउनुहोस्।

जाँच: अघि/पछि measurement र log example सुरक्षित छन्।

DAY 104 User trial

पढ्ने: Task-based feedback: काम पूरा भयो कि भएन, कहाँ अड्कियो।

बनाउने: २-३ उपलब्ध व्यक्तिलाई ३ काम दिनुहोस्; उपलब्ध नभए नयाँ फोनमा self-test record गर्नुहोस्।

जाँच: Feedback वा self-test भनेर सही label छ; top ३ issues तय छन्।

DAY 105 Release candidate

पढ्ने: Bug triage, changelog, screenshots र limitations।

बनाउने: Top issues समाधान; README, setup, demo र versioned release candidate बनाउनुहोस्।

जाँच: Installable candidate र known-issues list तयार छन्।

अर्को हप्ता सुरु गर्ने मापदण्ड

मुख्य flows test छन्; offline/expiry/permission cases pass; user feedback लागू भएको छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्याउनुपर्ने Day: _____

WEEK 16 / DAYS 106-112 / 21 HOURS

Build, updates, CI/CD र release

App package, staged update र store submission को पूरा प्रक्रिया बुझ्ने।

पढाइका स्रोत: [R25 EAS Build setup](#) | [R26 EAS Update runtime versions](#) | [R27 EAS Workflows](#) | [R28 Submit to app stores](#)

दैनिक ३ घण्टा: 15m सम्झना + 35m पढाइ + 100m code + 20m जाँच + 10m log

DAY 106 App identity र variants

पढ्ने: Icon, splash, package/bundle id, app version; development/preview/production।

बनाउने: App config र EAS profiles मिलाउनुहोस्; dev/prod identifiers र API URLs अलग गर्नुहोस्।

जाँच: गलत backend मा production data जाने configuration छैन।

DAY 107 Production builds

पढ्ने: Signing credentials, Android APK/AAB र iOS cloud build को भूमिका।

बनाउने: Android production AAB र testing APK तयार गर्नुहोस्; credentials सुरक्षित राख्नुहोस्।

जाँच: Testing build फोनमा install हुन्छ; iOS परीक्षण भएको/नभएको छुट्टै लेखिएको छ।

DAY 108 EAS Update

पढ्ने: Runtime compatibility, channel/branch, JS/assets update र native rebuild।

बनाउने: Preview channel मा सानो text fix पठाउनुहोस्; rollback/recovery प्रक्रिया अभ्यास गर्नुहोस्।

जाँच: सही build ले update पाउँछ; native change लाई OTA-only समाधान मानिएको छैन।

DAY 109 CI/CD र EAS Workflows

पढ्ने: Workflow trigger, install, lint/test/build, secrets र manual promotion।

बनाउने: एउटा workflow configure गर्नुहोस्; test failure हुँदा release रोक्ने step राख्नुहोस्।

जाँच: Workflow run वा documented local validation छ; secrets repository मा छैनन्।

WEEK 16 / VisitBook v1 + portfolio evidence

बाँकी अभ्यास र साप्ताहिक जाँच

DAY 110 Google Play release तयारी

पढ्ने: Store listing, screenshots, privacy/data declarations, testing track र current console requirements।

बनाउने: Internal test/release checklist; listing text, screenshots र data inventory तयार गर्नुहोस्।

जाँच: User data र permissions अनुसार disclosures मिल्छन्; approval deadline वाचा गरिएको छैन।

DAY 111 iOS release र operating the app

पढ्ने: App Store Connect/TestFlight flow, device validation, crash report, support र backup।

बनाउने: iOS access भए test गर्नुहोस्; नभए स्पष्ट pending checklist बनाउनुहोस्। support/contact र bug reporting तय गर्नुहोस्।

जाँच: Android success लाई iOS-tested भनिएको छैन; maintenance जिम्मेवारी स्पष्ट छ।

DAY 112 Final assessment र portfolio

पढ्ने: Architecture, storage, auth, sync, tests, release र trade-offs को पुनरावृत्ति।

बनाउने: ५ मिनेट demo, repository README, screenshots र final build राख्नुहोस्; १० self-review प्रश्नको उत्तर लेख्नुहोस्।

जाँच: v1 artifact तयार छ; store submission/approval अलग status मा छ; अर्को सुधारको सूची छ।

अर्को हप्ता सुरु गर्ने मापदण्ड

Android release artifact र tested update तयार छन्; submission स्थिति इमानदारीपूर्वक लेखिएको छ।

साताको evidence: working code + screenshot/demo + README मा setup, feature, test र known issue।

सकिएको मिति: _____ दोहोर्‍याउनुपर्ने Day: _____

BUILD & KEEP

Project portfolio र अन्तिम app

Project	मुख्य प्रमाण
Profile / Contacts	Reusable components, form validation, FlatList/SectionList, readable UI।
Pokémon / News	API states, details, pagination/infinite scroll र failure recovery।
Notes Buddy	SQLite CRUD, restart persistence, export/import र attachments।
Device / Reminder Lab	Permissions, map/sensors, local reminder, push र tap navigation।
Weather / Wallpaper	Custom hook, location fallback, filters, image list र modal।
Story Pocket	Backend AI, local story storage, loading/retry र usage limits।
VisitBook capstone	Offline visits, photo/location, follow-up, account/sync, tests र release।

VisitBook: सानो तर पूरा MVP

ग्राहक भेट, service visit वा field work को note राख्ने app। नाम, note, photo, optional location, follow-up reminder र search/filter। Account र sync पछिको चरणमा जोडिन्छ। AI summary वा export मध्ये एउटा optional feature हो।

Capstone acceptance: ८ जाँच

१. Airplane mode मा visit save/edit/delete।
२. Restart पछि data बाँकी।
३. Camera/location permission नदिएँदा manual entry।
४. Reminder edit/delete हुँदा पुरानो schedule हट्ने।
५. Login भएका प्रयोगकर्ताको data अलग।
६. Reconnect/retry गर्दा duplicate र data loss नहुने।
७. कम्तीमा ५ meaningful automated tests र १० manual cases।
८. Installable Android build, README, screenshots र demo।

Portfolio मा तीन बलिया apps प्राथमिकता दिनुहोस्: Notes Buddy, Story Pocket र VisitBook। अरू साना प्रयोगलाई एउटै mobile-labs repository मा राख्न सकिन्छ।

COVERAGE / PART 1

तपाईंको course: विषय कहाँ समेटियो?

तपाईंले पठाएका lecture विषयलाई मिल्दो समूहमा राखिएको छ। एउटै अवधारणा दोहोरिने lecture एउटै अभ्यासमा समेटिएको छ।

Course विषय	Day / अध्ययन
Introduction, mobile career, architecture, Expo vs React Native	1, 6-7; 57; 90। Instructor story/code-download को ठाउँमा आफ्नै लक्ष्य र repository।
Expo version change, VS Code, Bun setup, Expo Go	6; setup पृष्ठ; 43।
View, Text, Image, TextInput, Pressable, props, StyleSheet	8-11; React revision 4-5।
FlatList, SectionList, dark profile gallery	12-14।
React Navigation, params, tabs, modals, drawer	15-16, 19-21।
Pokémon app, API, pagination, detail, infinite scroll	22-25, 28।
Icon र splash	106; development build मा फेरि जाँच।
Expo Router, dynamic, catch-all, unmatched, Slot, groups	17-20।
Stack, tabs, drawer र platform-specific code	15, 19-21।
Expo Router API routes र client calls	27; server hosting boundary पनि।
External API, NewsAPI र rendering errors	22-28।
AsyncStorage, SecureStore, SQLite, File System, Notes Buddy	29-35; photo files 38।

COVERAGE / PART 2

Course coverage र थप engineering

Course विषय	Day / अध्ययन
Accelerometer, gyroscope, compass, light sensor, pedometer, tilt game	44-45।
Camera, audio, images	36-40; image generation 82।
Network, Linking, Haptics, Battery	41, 50-51।
EAS, development build, Android cloud build, variants	43, 106-107।
Maps, camera, zoom, gestures, markers, polylines	46-49।
Local/push notifications र emoji reminder app	52-56।
Protected routes, SQLite auth lesson, Google/GitHub OAuth	59-66; SQLite-only login सिकाइ simulation भनेर स्पष्ट।
Better Auth TypeScript lesson र Clerk	67-70; मुख्य coding JavaScript।
EAS Update, CI/CD, Play Store release	108-112।
Weather API, custom hook, location, dynamic background	71-73।
Pixabay wallpaper, header/list, filters, image detail modal	74-77।
AI story, navigation, database बिना saving, text/image AI	78-84; local SQLite/JSON saving।
६ role plays र quiz	तलका ६ design exercises; अन्तिम self-review पृष्ठ।
थप advanced अभ्यास	85-91: sync, idempotency, tests, profiling, accessibility, native boundary।

६ design exercises: Day 7 architecture; Day 35 storage decision; Day 63 authentication; Day 84 AI experience; Day 110 production release; Day 112 mobile developer trade-offs। प्रत्येकमा ५-१० वाक्यको निर्णय note लेख्नुहोस्।

[CLICKABLE READING INDEX](#)

पढ़ने आधिकारिक स्रोत 1/4

सबै documentation पढ़िसक्ने लक्ष्य नराख्नुहोस्। आजको Day मा भएका concepts को १-२ sections पढ्नुहोस्, example प्रयोग गर्नुहोस् र आवश्यक परे reference मा फर्कनुहोस्।

R01 · JavaScript Guide - MDN

Functions, objects, arrays, promises र modules।

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

R02 · React Learn

Components, state, events, effects र custom hooks।

<https://react.dev/learn>

R03 · React Native documentation

Core Components, Style, Flexbox, FlatList, SectionList र Networking खोल्नुहोस्।

<https://reactnative.dev/docs/getting-started>

R04 · Expo project templates

blank JavaScript template, package manager र project creation।

<https://docs.expo.dev/more/create-expo/>

R05 · React Navigation

Fundamentals, params, stack, tabs र drawer।

<https://reactnavigation.org/docs/getting-started/>

R06 · Expo Router setup

Manual setup पछि Router Introduction, layouts, notation, dynamic routes र protected routes पढ्नुहोस्।

<https://docs.expo.dev/router/installation/>

R07 · Expo Router API routes

Client र server boundary, API handler र hosting requirement।

<https://docs.expo.dev/router/web/api-routes/>

R08 · Expo Router concepts

File routing र navigation को आधिकारिक परिचय।

<https://docs.expo.dev/router/introduction/>

R09 · AsyncStorage

साधारण persistent key-value data।

<https://docs.expo.dev/versions/latest/sdk/async-storage/>

R10 · SecureStore

सानो संवेदनशील data; platform behavior र limitations।

<https://docs.expo.dev/versions/latest/sdk/securestore/>

[CLICKABLE READING INDEX](#)

पढ़ने आधिकारिक स्रोत 2/4

R11 · SQLite

Local relational data, SQL queries, transactions र migration अभ्यास।

<https://docs.expo.dev/versions/latest/sdk/sqlite/>

R12 · Expo SDK reference

Camera, ImagePicker, Image, Audio, FileSystem, Haptics, Battery, Linking र Location search गर्नुहोस्।

<https://docs.expo.dev/versions/latest/>

R13 · Development builds

Expo Go, native libraries, cloud/local build र dev client।

<https://docs.expo.dev/develop/development-builds/introduction/>

R14 · Expo Maps

Current platform support र alpha status पहिले पढ्नुहोस्; आवश्यक भए SDK index बाट react-native-maps हेर्नुहोस्।

<https://docs.expo.dev/versions/latest/sdk/maps/>

R15 · Expo Sensors

Accelerometer, gyroscope, magnetometer, light र pedometer links।

<https://docs.expo.dev/versions/latest/sdk/sensors/>

R16 · Network and Linking - SDK index

Network/NetInfo र Linking entries; AppState का लागि React Native docs।

<https://docs.expo.dev/versions/latest/>

R17 · Notifications

Local schedules, remote push, permissions; linked push setup guide पनि खोल्नुहोस्।

<https://docs.expo.dev/versions/latest/sdk/notifications/>

R18 · OAuth / OpenID authentication

Scheme, redirects, PKCE र provider examples; development build प्रयोग।

<https://docs.expo.dev/guides/authentication/>

R19 · Expo environment variables

EXPO_PUBLIC values app bundle मा देखिन्छन्; server secrets अलग राख्नुहोस्।

<https://docs.expo.dev/guides/environment-variables/>

R20 · Better Auth Expo integration

Server/client integration र secure storage; version अनुसार examples मिलाउनुहोस्।

<https://better-auth.com/docs/integrations/expo>

[CLICKABLE READING INDEX](#)

पढ़ने आधिकारिक स्रोत 3/4

R21 · Clerk Expo quickstart

Provider, session, token cache र न्यूनतम working flow।

<https://clerk.com/docs/expo/getting-started/quickstart>

R22 · Expo unit testing + RN testing

Jest setup; testing layers का लागि reactnative.dev/docs/testing-overview पनि पढ़नुहोस्।

<https://docs.expo.dev/develop/unit-testing/>

R23 · React Native performance

Release build, render work, profiling र bottleneck-driven fixes।

<https://reactnative.dev/docs/performance>

R24 · React Native accessibility

Labels, roles, focus, screen readers र accessible controls।

<https://reactnative.dev/docs/accessibility>

R25 · EAS Build setup

Profiles, credentials, native builds र store-ready artifacts।

<https://docs.expo.dev/build/setup/>

R26 · EAS Update runtime versions

Compatibility, native changes र update rollout boundaries।

<https://docs.expo.dev/eas-update/runtime-versions/>

R27 · EAS Workflows

CI/CD workflow configuration, build/test automation र triggers।

<https://docs.expo.dev/eas/workflows/get-started/>

R28 · Submit to app stores

Android/iOS submission प्रक्रिया; store review अलग चरण हो।

<https://docs.expo.dev/deploy/submit-to-app-stores/>

R29 · Expo Audio

Playback र recording को current API; पुरानो lesson को API सँग version जाँच्नुहोस्।

<https://docs.expo.dev/versions/latest/sdk/audio/>

[CLICKABLE READING INDEX](#)

पढ़ने आधिकारिक स्रोत 4/4

R30 · Express backend

Day 57-63: server setup, routing, middleware र error handling पढ़नुहोस्।

<https://expressjs.com/en/starter/hello-world/>

R31 · PokéAPI reference

Day 22-25: pagination र Pokémon/detail response को schema।

<https://pokeapi.co/docs/v2>

R32 · NewsAPI documentation

Day 26: API key, endpoints, errors र आफ्नो plan को usage conditions।

<https://newsapi.org/docs>

R33 · Open-Meteo weather API

Day 71-73: weather API को एउटा विकल्प; coordinates, units र response fields।

<https://open-meteo.com/en/docs>

R34 · Pixabay API

Day 74-76: search filters, pagination र image use requirements।

<https://pixabay.com/api/docs/>

R35 · React Native Testing Library

Day 70, 87, 99: user behavior, screen queries र component interaction tests।

<https://oss.callstack.com/react-native-testing-library/>

Sources जाँच मिति: 26 September 2026। Docs का "latest" पृष्ठ बदलिन सक्छन्। Implementation गर्दा आफ्नो installed Expo SDK र package version मिलाउनुहोस्।

पढ़ने ठ्याक्कै सीमा

प्रत्येक दिन ३५ मिनेटमा कामसँग सम्बन्धित १-२ sections। उदाहरणका लागि Day 12 मा FlatList को data/renderItem/keyExtractor; Day 31 मा SQLite को database खोल्ने र query गर्ने भाग। पूरै API reference एकैपटक पढ़नु पर्दैन।

State management: सुरुमा useState/Context पुग्छ। Day 69 मा global client state आवश्यक भएको feature मा Zustand प्रयोग गर्नुहोस्। Server data cache र local UI state को जिम्मेवारी छुट्टै बुझ्नुहोस्।

MARK AFTER YOU BUILD

तपाईंको progress tracker

हरेक day पूरा भएपछि D मा टिक गर्नुहोस्; weekly checkpoint पुगेपछि Gate मा टिक गर्नुहोस्। वास्तविक पढेको घण्टा पनि टिप्नुहोस्।

W	Day	D1 D2 D3 D4 D5 D6 D7	Gate	घण्टा
01	1-7	[][][][][][][]	[]	—
02	8-14	[][][][][][][]	[]	—
03	15-21	[][][][][][][]	[]	—
04	22-28	[][][][][][][]	[]	—
05	29-35	[][][][][][][]	[]	—
06	36-42	[][][][][][][]	[]	—
07	43-49	[][][][][][][]	[]	—
08	50-56	[][][][][][][]	[]	—
09	57-63	[][][][][][][]	[]	—
10	64-70	[][][][][][][]	[]	—
11	71-77	[][][][][][][]	[]	—
12	78-84	[][][][][][][]	[]	—
13	85-91	[][][][][][][]	[]	—
14	92-98	[][][][][][][]	[]	—
15	99-105	[][][][][][][]	[]	—
16	106-112	[][][][][][][]	[]	—

हरेक project को छोटो log

Copy गर्ने ढाँचा

Day / मिति: _____

आजको लक्ष्य: _____

आफैं बनाएको feature: _____

आएको error र समाधान: _____

जाँचेको failure case: _____

भोलीको पहिलो काम: _____

EXPLAIN, BUILD, VERIFY

अन्तिम self-review

यी प्रश्नको उत्तर code/diagram/demo सहित आफ्नै शब्दमा दिनुहोस्। उत्तर अस्पष्ट भए सम्बन्धित Day फेरि अभ्यास गर्नुहोस्।

- 01. React Native र Expo कसरी सम्बन्धित छन्? Expo Go बाट dev build किन जानुपर्छ?
- 02. State, props, effect र event handler मा data flow कसरी हुन्छ?
- 03. १०,००० items भए list, image र network load कसरी जाँच्नुहुन्छ?
- 04. Theme, session, notes र photos कुन storage मा राख्नुहुन्छ र किन?
- 05. Protected screen भए पनि server authorization किन चाहिन्छ?
- 06. OAuth redirect, session restore र expired token को flow के हो?
- 07. Offline note reconnect पछि दोहोरिएन भनेर कसरी सुनिश्चित गर्नुहुन्छ?
- 08. AI provider key कहाँ राख्नुहुन्छ? timeout र usage limit कसले लागू गर्छ?
- 09. कुन परिवर्तन EAS Update बाट जान्छ? कुन परिवर्तनमा native build चाहिन्छ?
- 10. App release भएपछि crash, user feedback र data समस्या कसरी सम्हाल्नुहुन्छ?

अगाडि बढ्ने क्षमता

Stage	देखाउनुपर्ने क्षमता
Day 28	Multi-screen API app आफैं बनाउने।
Day 56	Offline data र device permissions सहित app चलाउने।
Day 84	Auth/backend र AI feature end to end जोड्ने।
Day 112	Test, sync र release evidence सहित आफ्नो product प्रस्तुत गर्ने।

Advanced दक्षता बढाउने अर्को काम: ३० दिन वास्तविक प्रयोगकर्ता feedback अनुसार एउटै app सुधारनुहोस्। काम गर्ने feature, user feedback र तपाईंको व्याख्याले प्रगति देखाउँछन्।

KEEP AS REFERENCE

छोटो शब्दकोश र त्यसपछिको बाटो

शब्द	सजिलो अर्थ
Component	UI को पुनः प्रयोग गर्न मिल्ने भाग।
State / props	State बदलिने आफ्नै data; props अर्को component बाट आएको data।
Route / layout	कुन screen खुल्छ भन्ने नियम / screens को साझा संरचना।
API / backend	App ले request पठाउने interface / server मा चल्ने logic।
Persistence	App बन्द गरेपछि पनि data बचिरहनु।
Authentication / authorization	तपाईं को हो? / तपाईंलाई यो काम गर्न अनुमति छ?
Development build	आफ्नो native configuration र developer tools भएको app build।
Native module / plugin	OS feature सँग जोडिने code / native configuration मिलाउने व्यवस्था।
Idempotency / sync	Request दोहोरिँदा duplicate effect नहुने / local-server data मिलाउने।
OTA / CI/CD	Compatible JS/assets update / automated checks, builds र release workflow।

Optional specialization: आफ्नो लक्ष्य अनुसार

१४ दिन: TypeScript बिस्तारै अपनाउने वा testing गहिरो बनाउने।

१४-२१ दिन: Realtime features, background tasks र ठूलो offline dataset।

२१-३० दिन: Native modules, complex animations वा app performance।

३० दिन: आफ्नै product मा user interviews, fixes, release र monitoring।

पहिलो कदम

Day 1 को तीन JavaScript काम र आफ्नो mobile product लक्ष्य लेख्नुहोस्। त्यसपछि यही PDF को क्रमसँगै अध्ययन गर्नुहोस्। हरेक दिन सानो working result बनाएर अगाडि बढ्नुहोस्।